

Earning trust: privacy, observability and control from the user's side

October 2026 · LocalAI Agent 0.15.0

1. Why people don't use Muse or Instinct

Launch coverage and reviews of both products return to the same worries. The capabilities impress people; handing them over does not feel safe.

Worry	What people said or saw	What it means for us
"My whole life sits on their server."	Muse is cloud-only and US-only, with a per-user VM. Instinct reportedly kept data after people disconnected accounts.	Local storage is our biggest advantage, but only if users <i>believe</i> it. They need to see it, not just read it in a privacy policy.
"I don't know what it's doing."	Agents act in the background, and activity logs are terse.	Every read, every action and every byte that leaves must be visible and explained.
"It might do something I can't undo."	Instinct's terms let it enter binding agreements , and testers reported an unapproved action .	Hard limits that no setting can lift. Approvals that show exactly what will happen. Undo where possible.
"Someone else can make it act."	A prompt injection via email in Instinct's beta.	A visible, explainable defence: "this email tried to instruct me; I ignored it".
"It asks for everything up front."	Connect email, calendar, messages, payments... at sign-up.	Ask for each permission only when a feature needs it, explain why, and allow saying no.
"I can't turn just one thing off."	It's all or nothing.	One switch per capability, with clear effects, including "forget what you learned from it".

2. What the agent asks for today

Access	OS permission / grant	Needed for	What is read	Default
Messages (iMessage/SMS)	Full Disk Access (macOS)	Reading chats, waiting-on-you nudges, iMessage channel	<code>chat.db</code> , read-only	on (works once FDA is granted)
WhatsApp	Full Disk Access	Reading WhatsApp chats	<code>ChatStorage.sqlite</code> , read-only	on
Calendar	Calendars: Full Access (EventKit) or Automation → Calendar	Agenda, scheduling, brief, free/busy for friends	events in range	on
Reminders, Notes, Mail, Contacts	Automation per app	Each connector	as asked	on
Mac apps	Accessibility	Pressing buttons, typing in apps	the front window's UI tree	on

Access	OS permission / grant	Needed for	What is read	Default
Screen context	Screen Recording	"What am I looking at?"	screenshots → on-device OCR text, kept 2 h	off
Microphone	Browser mic permission	Voice, wake word	audio in memory only	off until used
Files	none (allowed folder list)	Find, move, open, trash	names; contents when asked	Desktop, Documents, Downloads
Gmail	Google OAuth (your own client)	Gmail tools	mail	off until signed in
Browser	its own Chrome profile	Web tasks	pages it opens	on
Cloud model	your Anthropic key	"Think harder"	what the approval card lists	off
Trusted agents	a network listener (:8766)	Friends' agents	your free/busy (if allowed)	off
Phone line	Twilio + tunnel	Calls	your speech (via Twilio)	off
iMessage channel	Messages	Texting the agent	your messages to yourself	off

Three problems with the current state:

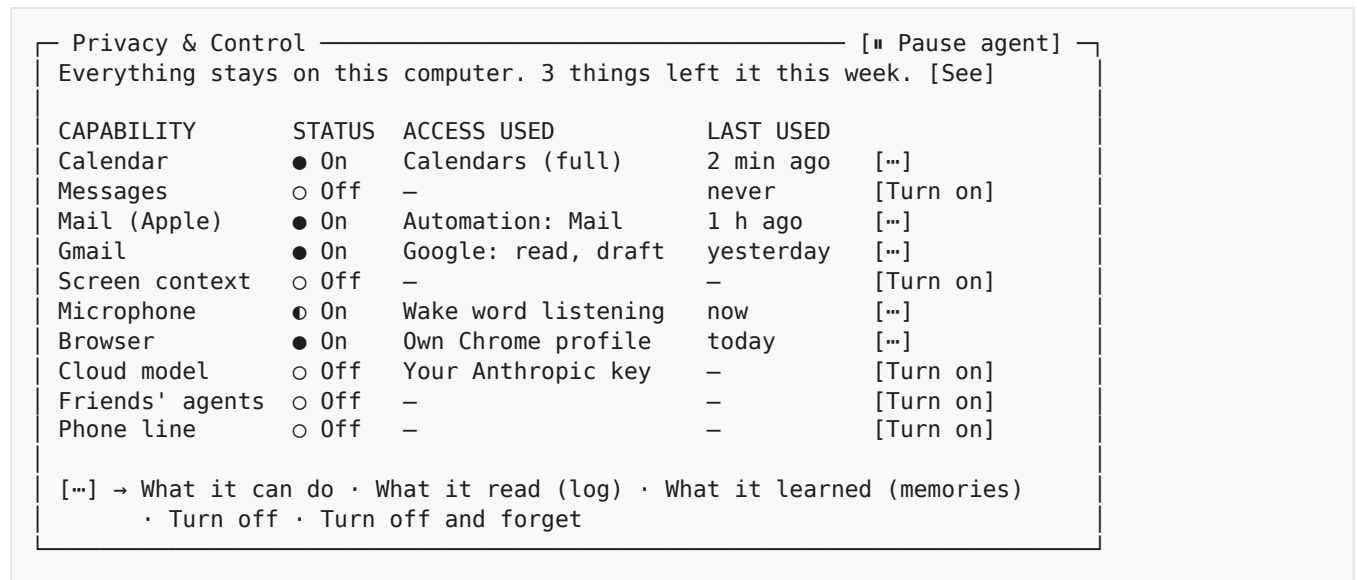
1. **Too much is on by default.** Messages, WhatsApp, Mail, Contacts and Mac apps are enabled before the user has decided they want them, and they quietly start working once a permission happens to exist.
2. **OS permissions go to Terminal or Python, not to "LocalAI Agent".** System Settings shows "Terminal" with Full Disk Access, which is confusing and grants more than intended (see the security review, P1-7).
3. **There's no single place to see and control all of this.** Settings has many fieldsets and the Connectors tab shows status, but there's no one view that says "here is everything I can access, and here is what I did with it".

3. Design principles for a trustworthy agent

1. **Off until you turn it on.** Every capability that reads personal data starts off. Setup is a short guided "what should I help with?" that turns on only what the user picks.
2. **Ask at the moment of need, in plain words.** The first time the user asks about their messages, the agent explains: "To read your messages I need Full Disk Access. I'll only read chats when you ask, or for 'waiting on you' nudges if you enable them. Nothing leaves this computer." Then [Open System Settings] or [Not now].
3. **Show, don't claim.** A live indicator for anything sensitive happening right now, and a ledger of everything that left the computer.
4. **Every action and nudge can answer "why?":** the decision probabilities, the sources it used, the rule that allowed it.
5. **Easy off, real off.** Turning a capability off stops it immediately and offers to delete what was learned from it.
6. **Limits no setting can lift.** No binding agreements, no payments without the user, no sending money, nothing irreversible by phone.
7. **Approval fatigue is a safety bug.** Fewer, better approvals: they show the exact payload, scope sensibly, and are never needed for harmless reads.

4. What to build

4.1 Privacy Control Center (one screen)



- **One row per capability.** It shows status, the OS permission behind it (with a deep link to the exact System Settings pane), and when it was last used.
- **Pause agent.** One click stops the scheduler, the listeners, the wake word and any running task. It is also exposed as `localagent pause` and the menu-bar icon.
- **Turn off and forget.** It disables the capability and deletes memories whose source was that capability (memories already record `source_message_id`; add `source_capability`). It also deletes cached data such as screen text and Gmail tokens.
- **Trust presets** at the top:
 - **Observer:** read only, no actions;
 - **Assistant:** drafts, never sends;
 - **Agent:** acts with approvals.

These presets map directly onto the policy tiers, so the user picks a mental model instead of 40 toggles.

4.2 Live indicators (like the camera light)

A small status strip in the app, mirrored in the menu bar or system tray:

- 🗣️ **Listening** for the wake word / recording.
- 👁️ **Screen context** captured N seconds ago.
- 🌐 **Browser** is acting (the click overlay is already shown in the page).
- ☁️ **Sending to the cloud** (only during an approved cloud request).
- 📞 **On a call** / 🤖 **friend's agent connected.**

Each indicator is clickable and leads to "what is it doing right now, and stop it".

4.3 Activity that explains, not just records

The audit log already records every tool call, approval, refusal and flagged injection. Make it readable and answer three questions:

- **What did you read?** Log reads as well as actions, in plain language: "Read 3 iMessage threads (Mom, Priya, Sam) to answer 'anything I need to reply to?'"'. Background jobs should list their sources: "Morning brief read: calendar (4 events), reminders (2), Mail inbox (12 unread)".
- **Why did you do that?** Every action, nudge and message links to its *explanation card*:
 - the request that started it;
 - the decision chips with probabilities ("task 94%, needs approval because *write* tier");
 - the sources used, quoted;
 - the rule that let it run ("you allowed `calendar_create_event` for 1 hour at 10:02").
- **What left this computer?** An **egress ledger**, one row per outbound transfer:
 - cloud model requests, with the exact text sent;
 - Gmail and iMessage sends;
 - web searches and page visits (domain and query);
 - peer messages;
 - phone answers spoken via Twilio.

This is the single strongest proof of "local": a week with three rows says more than any policy.

Filters: by capability, by person mentioned, "only things that left", "only things I approved". The weekly **privacy receipt** is a one-screen summary (and optional notification): "This week: 212 reads, 9 actions you approved, 3 things sent (2 emails, 1 cloud question). 1 suspicious email ignored."

4.4 Approvals people can actually judge

- **Show the payload, not a summary.** The full email or message body, every recipient (cc/bcc), the full URL, the exact text going to the cloud. For long content, show the first lines with a "show all" that is open by default for sends.
- **Say what's irreversible.** "Sending can't be undone" vs "You can delete this event later".
- **Say why it needs approval.** "This sends data to someone else" (write) / "This can't be undone" (danger) / "This run read an email that tried to instruct me, so I'm asking again".
- **Suggest the narrowest sensible scope.** "Allow sending to Priya for 1 hour" rather than "Always". Never offer "Always" for new recipients.
- **Approve from where you are**, with the same full payload: the app, a notification action, or an iMessage reply (which must include the payload, not just the title).

4.5 Memory you can see and shape

This already exists: memories are listed and editable, `about-me.md` is a plain file, and the dream job reports what it learned. Add:

- **Source and confidence on each memory** ("from your message on 3 Oct"), and "never remember things like this".
- **Sensitive categories off by default:** health, finances, religion, sexuality, other people's private details. They are detected by the decision layer and kept only if the user confirms.
- **"What do you know about me?"** answered as a readable profile, grouped by topic, each item deletable.

4.6 Explaining nudges and decisions

Every nudge has a "Why am I seeing this?":

- "Priya asked you a question 2 days ago and you haven't replied (should-nudge 91%, urgency 3/5)".

- Controls: "Less like this", "Never for this person", "Not during work".

These are corrections that feed the decision layer, so the explanation doubles as training.

4.7 Onboarding that builds trust in minutes

1. "Everything I do happens on this computer. Here's how to check" (links to the egress ledger).
2. "What should I help with?" offers checkboxes for calendar, email, messages, files, web. Only those get turned on, each with its just-in-time OS prompt.
3. **First action is a draft, never a send**, so the user sees the approval card on something harmless.
4. **Demo of the injection guard**: a sample email with a hidden instruction, showing "I ignored this and told you".

5. Gap analysis: today vs target

Area	Today (0.15.0)	Target
Defaults	Most Apple connectors on	Everything personal off until chosen
Permissions	Granted to Terminal/Python; doctor shows status	Signed app identity; just-in-time prompts with purpose; deep links
Single control surface	Settings fieldsets + Connectors tab	Privacy Control Center with presets, pause, turn off and forget
Live status	Browser click overlay; voice indicator	Indicator strip + menu-bar icon for mic, screen, browser, cloud, call
Activity	Hash-chained log of actions, approvals, jobs	Reads logged in plain language; "why?" card per item; egress ledger ; weekly receipt
Approvals	Summary + collapsed details; 6 scopes	Full payload, irreversibility, reason, narrow default scope; text approvals with payload
Memory	Editable list + <code>about-me.md</code>	Source, sensitive-category gate, per-capability purge
Nudges	Decision chips available	"Why am I seeing this?" plus one-tap feedback
Kill switch	<code>localagent stop</code>	Pause button (app, menu bar, CLI, iMessage "pause")

6. How we'll know it works

Signal	Target
Users who enable at least 3 capabilities in week 1	> 60%
Users who open Activity or the egress ledger in week 1	> 40%
Approval acceptance rate	70–90% (too high means rubber-stamping; too low means bad proposals)
"Turn off" after enabling, within 7 days	< 10% per capability
Survey: "I understand what it can access" / "I feel in control"	≥ 4.5 / 5